

wire-shark - Phase-5 refine guide

Phase-5 refine

```
void layer_by_layer(const u_char *pkt, int plen) {
    if (!pkt || plen <= 0) {
        printf("[layer_by_layer] empty packet\n");
        return;
    }

    char extra[256];

    printf("\n==== Layer-by-Layer Analysis =====\n");
    printf("Total length: %d bytes\n\n", plen);

    const int eth_len = 14;
    if (plen < eth_len) {
        printf("[L2] Truncated Ethernet header (packet too small)\n");
        print_field_bytes("Raw Frame", pkt, plen, 0, plen - 1, "");
        return;
    }

    // ----- L2: Ethernet -----
    {
        char macbuf[64];
        snprintf(macbuf, sizeof(macbuf),
            "DST MAC
%02X:%02X:%02X:%02X:%02X:%02X",
            pkt[0], pkt[1], pkt[2], pkt[3], pkt[4], pkt[5]);
        print_field_bytes("Ethernet - Destination MAC", pkt,
            plen, 0, 5, macbuf);
    }
    {
        char macbuf[64];
        snprintf(macbuf, sizeof(macbuf),
            "SRC MAC
%02X:%02X:%02X:%02X:%02X:%02X",
            pkt[6], pkt[7], pkt[8], pkt[9], pkt[10], pkt[11]);
        print_field_bytes("Ethernet - Source MAC", pkt,
```

```

plen, 6, 11, macbuf);
    }

    uint16_t ethertype = get_u16_be(pkt, 12, plen);
    {
        char etbuf[64];
        snprintf(etbuf, sizeof(etbuf), "EtherType 0x%04X",
ethertype);
        print_field_bytes("Ethernet - EtherType", pkt, plen,
12, 13, etbuf);
    }

    // ----- L3: IPv6 -----
    if (ethertype == ETHERTYPE_IPV6) {
        int ip6_off = eth_len;
        if (ip6_off + 40 > plen) {
            printf("[L3 IPv6] Header truncated\n");
            return;
        }

        uint32_t vtf = ntohl(*(uint32_t *)(pkt + ip6_off));
        int version = (vtf >> 28) & 0xF;
        int traffic_class = (vtf >> 20) & 0xFF;
        int flow_label = vtf & 0xFFFF;

        snprintf(extra, sizeof(extra),
            "Version=%d | TrafficClass=0x%02X |
FlowLabel=0x%05X",
            version, traffic_class, flow_label);
        print_field_bytes("IPv6 - Version/TC/FlowLabel",
pkt, plen, ip6_off, ip6_off + 3, extra);

        uint16_t payload_len = get_u16_be(pkt, ip6_off + 4,
plen);
        uint8_t next_hdr = pkt[ip6_off + 6];
        uint8_t hop = pkt[ip6_off + 7];

        snprintf(extra, sizeof(extra),
            "PayloadLen=%u | NextHeader=%u |
HopLimit=%u",
            payload_len, next_hdr, hop);
        print_field_bytes("IPv6 -

```

```

PayloadLen/Next/HopLimit", pkt, plen, ip6_off + 4,
ip6_off + 7, extra);

    char srcbuf[INET6_ADDRSTRLEN],
dstbuf[INET6_ADDRSTRLEN];
    inet_ntop(AF_INET6, pkt + ip6_off + 8, srcbuf,
sizeof(srcbuf));
    inet_ntop(AF_INET6, pkt + ip6_off + 24, dstbuf,
sizeof(dstbuf));

    snprintf(extra, sizeof(extra), "Src %s", srcbuf);
    print_field_bytes("IPv6 - Source Address", pkt, plen,
ip6_off + 8, ip6_off + 23, extra);
    snprintf(extra, sizeof(extra), "Dst %s", dstbuf);
    print_field_bytes("IPv6 - Destination Address", pkt,
plen, ip6_off + 24, ip6_off + 39, extra);

    int l4_off = ip6_off + 40;
    if (next_hdr == IPPROTO_UDP && l4_off + 8 <=
plen) {
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        uint16_t ulen = get_u16_be(pkt, l4_off + 4, plen);
        uint16_t chksum = get_u16_be(pkt, l4_off + 6,
plen);

        snprintf(extra, sizeof(extra),
                "SrcPort=%u | DstPort=%u | Length=%u |
Checksum=0x%04X",
                sport, dport, ulen, chksum);
        print_field_bytes("UDP (IPv6) - Header", pkt,
plen, l4_off, l4_off + 7, extra);

        if (l4_off + 8 < plen) {
            snprintf(extra, sizeof(extra), "UDP Payload (%d
bytes)", plen - (l4_off + 8));
            print_field_bytes("L7 - Payload (UDP/IPv6)",
pkt, plen, l4_off + 8, plen - 1, extra);
        }
    }
    // (TCP over IPv6 can be added similarly)

```

```

    } else if (ethertype == ETHERTYPE_IP) {
        // keep your existing IPv4 parser here (unchanged)
        ...
    } else if (ethertype == ETHERTYPE_ARP) {
        // keep your existing ARP parser here (unchanged)
        ...
    } else {
        print_field_bytes("L3 - Unknown EtherType", pkt,
plen, eth_len, plen - 1, "Not parsed");
    }

    printf("==== End of layer-by-layer analysis
====\n\n");
}

```

complete the pretty print layer_by_layer function taking inspiration from the ugly print one below:

```

void layer_by_layer(const u_char *pkt, int plen) {
    if (!pkt || plen <= 0) {
        printf("[layer_by_layer] empty packet\n");
        return;
    }

    char extra[256];

    printf("\n==== Layer-by-Layer Analysis ==== \n");
    printf("Total length: %d bytes\n\n", plen);

    // ----- L2: Ethernet -----
    //const int eth_off = 0;
    const int eth_len = 14;
    if (plen < eth_len) {
        printf("[L2] Truncated Ethernet header (packet too
small)\n");
        print_field_bytes("Raw Frame", pkt, plen, 0, plen -
1, "");
        return;
    }

    // Destination MAC (0-5)

```

```

{
    char macbuf[64];
    snprintf(macbuf, sizeof(macbuf),
        "DST MAC
%02X:%02X:%02X:%02X:%02X:%02X",
        pkt[0], pkt[1], pkt[2], pkt[3], pkt[4], pkt[5]);
    print_field_bytes("Ethernet - Destination MAC", pkt,
plen, 0, 5, macbuf);
}
// Source MAC (6-11)
{
    char macbuf[64];
    snprintf(macbuf, sizeof(macbuf),
        "SRC MAC
%02X:%02X:%02X:%02X:%02X:%02X",
        pkt[6], pkt[7], pkt[8], pkt[9], pkt[10], pkt[11]);
    print_field_bytes("Ethernet - Source MAC", pkt,
plen, 6, 11, macbuf);
}
// EtherType (12-13)
uint16_t ethertype = get_u16_be(pkt, 12, plen);
{
    char etbuf[64];
    snprintf(etbuf, sizeof(etbuf), "EtherType 0x%04X",
ethertype);
    print_field_bytes("Ethernet - EtherType", pkt, plen,
12, 13, etbuf);
}

// Move to L3 depending on ethertype
if (ethertype == ETHERTYPE_IP) {
    // ----- L3: IPv4 -----
    int ip_off = eth_len;
    if (ip_off + 20 > plen) {
        printf("[L3 IPv4] Header truncated or not
present\n\n");
        return;
    }
    uint8_t ver_ihl = pkt[ip_off];
    uint8_t version = ver_ihl >> 4;
    uint8_t ihl = (ver_ihl & 0x0F) * 4;
    uint8_t tos = pkt[ip_off + 1];

```

```

uint16_t totlen = get_u16_be(pkt, ip_off + 2, plen);
uint16_t id = get_u16_be(pkt, ip_off + 4, plen);
uint16_t frag = get_u16_be(pkt, ip_off + 6, plen);
uint8_t ttl = pkt[ip_off + 8];
uint8_t proto = pkt[ip_off + 9];
uint16_t checksum = get_u16_be(pkt, ip_off + 10,
plen);

uint32_t srcip = get_u32_be(pkt, ip_off + 12, plen);
uint32_t dstip = get_u32_be(pkt, ip_off + 16, plen);

char extra[256];
snprintf(extra, sizeof(extra),
          "IPv%d IHL=%d bytes | TOS=0x%02X |
TotalLen=%u",
          version, ihl, tos, totlen);
print_field_bytes("IPv4 - Version/IHL/TOS/Len (first
byte..3)", pkt, plen, ip_off, ip_off + 3, extra);

snprintf(extra, sizeof(extra),
          "ID=0x%04X | Flags+FragOff=0x%04X", id,
frag);
print_field_bytes("IPv4 - ID/Flags/FragOffset", pkt,
plen, ip_off + 4, ip_off + 7, extra);

snprintf(extra, sizeof(extra),
          "TTL=%u | Protocol=%u |
HeaderChecksum=0x%04X", ttl, proto, checksum);
print_field_bytes("IPv4 - TTL/Protocol/Checksum",
pkt, plen, ip_off + 8, ip_off + 11, extra);

// src/dst IPs
{
    struct in_addr a;
    a.s_addr = htonl(srcip); // we've read bytes
directly, convert to network order before inet_ntop
    char ssrc[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &a, ssrc, sizeof(ssrc));
    snprintf(extra, sizeof(extra), "Src IP %s", ssrc);
    print_field_bytes("IPv4 - Source IP", pkt, plen,
ip_off + 12, ip_off + 15, extra);
}
{

```

```

    struct in_addr a;
    a.s_addr = htonl(dstip);
    char sdst[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &a, sdst, sizeof(sdst));
    snprintf(extra, sizeof(extra), "Dst IP %s", sdst);
    print_field_bytes("IPv4 - Destination IP", pkt,
    plen, ip_off + 16, ip_off + 19, extra);
    }

    // Options (if any)
    if (ihl > 20) {
        int opt_start = ip_off + 20;
        int opt_end = ip_off + ihl - 1;
        print_field_bytes("IPv4 - Options", pkt, plen,
    opt_start, opt_end, "IP options (if present)");
    }

    // ----- L4: TCP/UDP -----
    int l4_off = ip_off + ihl;
    if (l4_off >= plen) {
        printf("[L4] No L4 header available (packet ends
    at L3)\n\n");
        return;
    }

    if (proto == IPPROTO_TCP) {
        if (l4_off + 20 > plen) {
            printf("[TCP] Truncated TCP header\n\n");
            return;
        }
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        uint32_t seq = get_u32_be(pkt, l4_off + 4, plen);
        uint32_t ack = get_u32_be(pkt, l4_off + 8, plen);
        uint8_t data_off_raw = pkt[l4_off + 12];
        uint8_t tcp_hdr_len = (data_off_raw >> 4) * 4;
        uint16_t win = get_u16_be(pkt, l4_off + 14, plen);
        uint16_t chksum = get_u16_be(pkt, l4_off + 16,
    plen);
        uint16_t urg = get_u16_be(pkt, l4_off + 18, plen);

        snprintf(extra, sizeof(extra), "Ports %u -> %u |

```

```

Seq=%u | Ack=%u", sport, dport, seq, ack);
    print_field_bytes("TCP - Ports/Seq/Ack (first 12
bytes)", pkt, plen, l4_off, l4_off + 11, extra);

    snprintf(extra, sizeof(extra), "DataOff=%u bytes |
Window=%u | Checksum=0x%04X | Urg=%u",
        tcp_hdr_len, win, chksum, urg);
    print_field_bytes("TCP -
DataOff/Flags/Window/Checksum/Urg", pkt, plen,
l4_off + 12, l4_off + 19, extra);

    // Flags interpretation (from bytes 12-13)
    if (l4_off + 13 < plen) {
        uint16_t flags_field = get_u16_be(pkt, l4_off +
12, plen);
        // flags in lower 9 bits usually
        char flags_str[128] = "";
        if (flags_field & TH_FIN) strcat(flags_str, "FIN ");
        if (flags_field & TH_SYN) strcat(flags_str, "SYN
");
        if (flags_field & TH_RST) strcat(flags_str, "RST ");
        if (flags_field & TH_PUSH) strcat(flags_str, "PSH
");
        if (flags_field & TH_ACK) strcat(flags_str, "ACK
");
        if (flags_field & TH_URG) strcat(flags_str, "URG
");
        if (flags_field & TH_ECE) strcat(flags_str, "ECE
");
        if (flags_field & TH_CWR) strcat(flags_str, "CWR
");
        if (flags_str[0] == '\0') strcpy(flags_str, "None");
        snprintf(extra, sizeof(extra), "Flags: %s
(0x%04X)", flags_str, flags_field & 0x1FF);
        print_field_bytes("TCP - Raw Flags", pkt, plen,
l4_off + 12, l4_off + 13, extra);
    }

    // Options (if any)
    if (tcp_hdr_len > 20) {
        int opt_start = l4_off + 20;
        int opt_end = l4_off + tcp_hdr_len - 1;

```



```

        print_field_bytes("TCP - Options", pkt, plen,
opt_start, opt_end, "TCP options");
    }

    // Payload
    int payload_start = l4_off + tcp_hdr_len;
    if (payload_start < plen) {
        int payload_end = plen - 1;
        snprintf(extra, sizeof(extra), "TCP Payload (%d
bytes)", plen - payload_start);
        print_field_bytes("L7 - Payload (TCP)", pkt,
plen, payload_start, payload_end, extra);
    } else {
        printf("L7 - Payload (TCP): None\n\n");
    }
} else if (proto == IPPROTO_UDP) {
    if (l4_off + 8 > plen) {
        printf("[UDP] Truncated UDP header\n\n");
        return;
    }
    uint16_t sport = get_u16_be(pkt, l4_off, plen);
    uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
    uint16_t ulen = get_u16_be(pkt, l4_off + 4, plen);
    uint16_t chksum = get_u16_be(pkt, l4_off + 6,
plen);

    snprintf(extra, sizeof(extra), "Ports %u -> %u |
Length=%u | Checksum=0x%04X", sport, dport, ulen,
chksum);
    print_field_bytes("UDP - Header", pkt, plen,
l4_off, l4_off + 7, extra);

    int payload_start = l4_off + 8;
    if (payload_start < plen) {
        int payload_end = plen - 1;
        snprintf(extra, sizeof(extra), "UDP Payload (%d
bytes)", plen - payload_start);
        print_field_bytes("L7 - Payload (UDP)", pkt,
plen, payload_start, payload_end, extra);
    } else {
        printf("L7 - Payload (UDP): None\n\n");
    }
}

```

```

    } else {
        // Unknown/Other L4
        print_field_bytes("L4 - Unknown/Other", pkt,
plen, l4_off, plen - 1, "Unrecognized L4 protocol");
    }

} else if (ethertype == ETHERTYPE_IPV6) {
    // ----- L3: IPv6 -----
    int ip6_off = eth_len;
    if (ip6_off + 40 > plen) {
        printf("[L3 IPv6] Header truncated or not
present\n\n");
        return;
    }

    // first 4 bytes: version (4b), tc (8b), flowlabel (20b)
    print_field_bytes("IPv6 - ver/tc/flowlabel", pkt, plen,
ip6_off, ip6_off + 3, "Version/TrafficClass/FlowLabel
(bytes 0-3)");
    uint16_t payload_len = get_u16_be(pkt, ip6_off + 4,
plen);
    uint8_t next_hdr = pkt[ip6_off + 6];
    uint8_t hop = pkt[ip6_off + 7];

    snprintf(extra, sizeof(extra), "PayloadLen=%u |
NextHeader=%u | HopLimit=%u", payload_len,
next_hdr, hop);
    print_field_bytes("IPv6 -
PayloadLen/Next/HopLimit", pkt, plen, ip6_off + 4,
ip6_off + 7, extra);

    // src/dst
    print_field_bytes("IPv6 - Source Addr", pkt, plen,
ip6_off + 8, ip6_off + 23, "128-bit src");
    print_field_bytes("IPv6 - Destination Addr", pkt,
plen, ip6_off + 24, ip6_off + 39, "128-bit dst");

    // L4 for IPv6: treat next_hdr similar to proto in
IPv4. IPv6 extension headers not fully parsed here.
    int l4_off = ip6_off + 40;
    if (l4_off >= plen) {
        printf("[L4] No L4 header available after IPv6

```

```

base header\n\n");
    return;
}
if (next_hdr == IPPROTO_TCP) {
    // reuse TCP parsing above but careful about
    bounds
    if (l4_off + 20 > plen) {
        printf("[TCP over IPv6] Truncated TCP
header\n\n");
        return;
    }
    uint16_t sport = get_u16_be(pkt, l4_off, plen);
    uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
    uint32_t seq = get_u32_be(pkt, l4_off + 4, plen);
    uint32_t ack = get_u32_be(pkt, l4_off + 8, plen);
    uint8_t data_off_raw = pkt[l4_off + 12];
    uint8_t tcp_hdr_len = (data_off_raw >> 4) * 4;
    snprintf(extra, sizeof(extra), "Ports %u -> %u |
Seq=%u | Ack=%u", sport, dport, seq, ack);
    print_field_bytes("TCP (IPv6) - Ports/Seq/Ack",
pkt, plen, l4_off, l4_off + 11, extra);
    snprintf(extra, sizeof(extra), "DataOff=%u bytes",
tcp_hdr_len);
    print_field_bytes("TCP (IPv6) -
Head/Flags/Window", pkt, plen, l4_off + 12, l4_off + 19,
extra);
    if (tcp_hdr_len > 20) print_field_bytes("TCP (IPv6)
- Options", pkt, plen, l4_off + 20, l4_off + tcp_hdr_len - 1,
"Options");
    if (l4_off + tcp_hdr_len < plen)
print_field_bytes("L7 - Payload (TCP/IPv6)", pkt, plen,
l4_off + tcp_hdr_len, plen - 1, "Payload");
    } else if (next_hdr == IPPROTO_UDP) {
        if (l4_off + 8 > plen) { printf("[UDP over IPv6]
Truncated UDP header\n\n"); return; }
        print_field_bytes("UDP (IPv6) - Header", pkt,
plen, l4_off, l4_off + 7,
"SrcPort/DstPort/Len/Checksum");
        if (l4_off + 8 < plen) print_field_bytes("L7 -
Payload (UDP/IPv6)", pkt, plen, l4_off + 8, plen - 1,
"Payload");
    } else {

```

```

        print_field_bytes("L4 - Unknown (IPv6)", pkt,
        plen, l4_off, plen - 1, "no parser for this NextHeader in
        this build");
    }

    } else if (ethertype == ETHERTYPE_ARP) {
        // ----- L3: ARP -----
        int arp_off = eth_len;
        if (arp_off + (int)sizeof(struct ether_arp) > plen) {
            print_field_bytes("ARP - truncated", pkt, plen,
            arp_off, plen - 1, "ARP header truncated");
            return;
        }
        // hardware type (2), proto type (2), hlen(1), plen(1),
        op(2), sha(6), spa(4), tha(6), tpa(4)
        uint16_t htype = get_u16_be(pkt, arp_off + 0, plen);
        uint16_t ptype = get_u16_be(pkt, arp_off + 2, plen);
        uint8_t hlen = pkt[arp_off + 4];
        uint8_t plen_sz = pkt[arp_off + 5];
        uint16_t op = get_u16_be(pkt, arp_off + 6, plen);
        char extra_arp[128];
        snprintf(extra_arp, sizeof(extra_arp),
        "HWType=0x%04X Proto=0x%04X HLen=%u PLen=%u
        Op=%u", htype, ptype, hlen, plen_sz, op);
        print_field_bytes("ARP - Header fields", pkt, plen,
        arp_off + 0, arp_off + 7, extra_arp);

        // SHA SPA
        print_field_bytes("ARP - Sender HW addr", pkt,
        plen, arp_off + 8, arp_off + 8 + hlen - 1, "Sender MAC");
        print_field_bytes("ARP - Sender Protocol addr", pkt,
        plen, arp_off + 8 + hlen, arp_off + 8 + hlen + plen_sz - 1,
        "Sender IP");
        int tha_off = arp_off + 8 + hlen + plen_sz;
        print_field_bytes("ARP - Target HW addr", pkt, plen,
        tha_off, tha_off + hlen - 1, "Target MAC");
        print_field_bytes("ARP - Target Protocol addr", pkt,
        plen, tha_off + hlen, tha_off + hlen + plen_sz - 1, "Target
        IP");

    } else {
        // Unknown EtherType — print remainder

```

```

        print_field_bytes("L3 - Unknown EtherType (raw
remainder)", pkt, plen, eth_len, plen - 1, "No parser for
EtherType");
    }

    printf("==== End of layer-by-layer analysis
====\n\n");
}

```

Here's a complete **pretty-print** `layer_by_layer` **function** for Phase 5, integrating the style from your "ugly print" and the more structured human-readable approach. It handles **Ethernet**, **IPv4**, **IPv6**, **ARP**, **TCP**, **UDP**, and prints payloads in a concise but readable manner:

```

c

void layer_by_layer(const u_char *pkt, int plen) {
    if (!pkt || plen <= 0) {
        printf("[layer_by_layer] empty packet\n");
        return;
    }

    char extra[256];

    printf("\n==== Layer-by-Layer Analysis =====\n");
    printf("Total length: %d bytes\n\n", plen);

    const int eth_len = 14;
    if (plen < eth_len) {
        printf("[L2] Truncated Ethernet header\n");
        print_field_bytes("Raw Frame", pkt, plen, 0, plen - 1, "");
        return;
    }

    // L2: Ethernet
    snprintf(extra, sizeof(extra),
        "DST MAC %02X:%02X:%02X:%02X:%02X:%02X",
        pkt[0], pkt[1], pkt[2], pkt[3], pkt[4], pkt[5]);
    print_field_bytes("Ethernet - Destination MAC", pkt, plen, 0, 5, extra);

    snprintf(extra, sizeof(extra),

```

```

        "SRC MAC %02X:%02X:%02X:%02X:%02X:%02X",
        pkt[6], pkt[7], pkt[8], pkt[9], pkt[10], pkt[11]);
print_field_bytes("Ethernet - Source MAC", pkt, plen, 6, 11, extra);

uint16_t ethertype = get_u16_be(pkt, 12, plen);
snprintf(extra, sizeof(extra), "EtherType 0x%04X", ethertype);
print_field_bytes("Ethernet - EtherType", pkt, plen, 12, 13, extra);

// L3: IPv4
if (ethertype == ETHERTYPE_IP) {
    int ip_off = eth_len;
    if (ip_off + 20 > plen) {
        printf("[L3 IPv4] Header truncated\n");
        return;
    }
    uint8_t ver_ihl = pkt[ip_off];
    uint8_t version = ver_ihl >> 4;
    uint8_t ihl = (ver_ihl & 0x0F) * 4;
    uint8_t tos = pkt[ip_off + 1];
    uint16_t totlen = get_u16_be(pkt, ip_off + 2, plen);
    uint16_t id = get_u16_be(pkt, ip_off + 4, plen);
    uint16_t frag = get_u16_be(pkt, ip_off + 6, plen);
    uint8_t ttl = pkt[ip_off + 8];
    uint8_t proto = pkt[ip_off + 9];
    uint16_t checksum = get_u16_be(pkt, ip_off + 10, plen);
    uint32_t srcip = get_u32_be(pkt, ip_off + 12, plen);
    uint32_t dstip = get_u32_be(pkt, ip_off + 16, plen);

    snprintf(extra, sizeof(extra),
        "IPv%d IHL=%d | TOS=0x%02X | TotalLen=%u", version, ihl, tos, totlen);
    print_field_bytes("IPv4 - Version/IHL/TOS/Len", pkt, plen, ip_off, ip_off + 3, extra);

    snprintf(extra, sizeof(extra),
        "ID=0x%04X | Flags+FragOffset=0x%04X", id, frag);
    print_field_bytes("IPv4 - ID/Flags/FragOffset", pkt, plen, ip_off + 4, ip_off + 7, extra);

    snprintf(extra, sizeof(extra),
        "TTL=%u | Protocol=%u | Checksum=0x%04X", ttl, proto, checksum);
    print_field_bytes("IPv4 - TTL/Protocol/Checksum", pkt, plen, ip_off + 8, ip_off + 11, extra);

    struct in_addr a;
    a.s_addr = htonl(srcip);

```

```

char ssrc[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &a, ssrc, sizeof(ssrc));
snprintf(extra, sizeof(extra), "Src IP %s", ssrc);
print_field_bytes("IPv4 - Source IP", pkt, plen, ip_off + 12, ip_off + 15, extra);

a.s_addr = htonl(dstip);
char sdst[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &a, sdst, sizeof(sdst));
snprintf(extra, sizeof(extra), "Dst IP %s", sdst);
print_field_bytes("IPv4 - Destination IP", pkt, plen, ip_off + 16, ip_off + 19, extra);

if (ihl > 20) print_field_bytes("IPv4 - Options", pkt, plen, ip_off + 20, ip_off + ihl - 1, "IP
Options");

// L4: TCP/UDP
int l4_off = ip_off + ihl;
if (proto == IPPROTO_TCP && l4_off + 20 <= plen) {
    uint16_t sport = get_u16_be(pkt, l4_off, plen);
    uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
    uint32_t seq = get_u32_be(pkt, l4_off + 4, plen);
    uint32_t ack = get_u32_be(pkt, l4_off + 8, plen);
    uint8_t data_off_raw = pkt[l4_off + 12];
    uint8_t tcp_hdr_len = (data_off_raw >> 4) * 4;
    uint16_t win = get_u16_be(pkt, l4_off + 14, plen);
    uint16_t chksum = get_u16_be(pkt, l4_off + 16, plen);

    snprintf(extra, sizeof(extra), "Ports %u -> %u | Seq=%u | Ack=%u", sport, dport, seq,
ack);
    print_field_bytes("TCP - Ports/Seq/Ack", pkt, plen, l4_off, l4_off + 11, extra);

    snprintf(extra, sizeof(extra), "DataOff=%u | Window=%u | Checksum=0x%04X",
tcp_hdr_len, win, chksum);
    print_field_bytes("TCP - DataOff/Win/Checksum", pkt, plen, l4_off + 12, l4_off + 19,
extra);

    if (tcp_hdr_len > 20)
        print_field_bytes("TCP - Options", pkt, plen, l4_off + 20, l4_off + tcp_hdr_len - 1, "TCP
Options");

    int payload_off = l4_off + tcp_hdr_len;
    if (payload_off < plen) {
        snprintf(extra, sizeof(extra), "TCP Payload (%d bytes)", plen - payload_off);
        print_field_bytes("L7 - Payload (TCP)", pkt, plen, payload_off, plen - 1, extra);
    }
}

```

```

    } else if (proto == IPPROTO_UDP && l4_off + 8 <= plen) {
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        uint16_t ulen = get_u16_be(pkt, l4_off + 4, plen);
        uint16_t checksum = get_u16_be(pkt, l4_off + 6, plen);

        snprintf(extra, sizeof(extra), "Ports %u -> %u | Length=%u | Checksum=0x%04X",
sport, dport, ulen, checksum);
        print_field_bytes("UDP - Header", pkt, plen, l4_off, l4_off + 7, extra);

        int payload_off = l4_off + 8;
        if (payload_off < plen) {
            snprintf(extra, sizeof(extra), "UDP Payload (%d bytes)", plen - payload_off);
            print_field_bytes("L7 - Payload (UDP)", pkt, plen, payload_off, plen - 1, extra);
        }
    }

} else if (ethertype == ETHERTYPE_IPV6) {
    int ip6_off = eth_len;
    if (ip6_off + 40 > plen) { printf("[L3 IPv6] truncated\n"); return; }

    uint32_t vtf = ntohl(*(uint32_t*)(pkt + ip6_off));
    int version = (vtf >> 28) & 0xF;
    int traffic_class = (vtf >> 20) & 0xFF;
    int flow_label = vtf & 0xFFFFF;
    snprintf(extra, sizeof(extra), "Version=%d | TC=0x%02X | FlowLabel=0x%05X", version,
traffic_class, flow_label);
    print_field_bytes("IPv6 - ver/tc/flowlabel", pkt, plen, ip6_off, ip6_off + 3, extra);

    uint16_t payload_len = get_u16_be(pkt, ip6_off + 4, plen);
    uint8_t next_hdr = pkt[ip6_off + 6];
    uint8_t hop = pkt[ip6_off + 7];
    snprintf(extra, sizeof(extra), "PayloadLen=%u | NextHeader=%u | HopLimit=%u",
payload_len, next_hdr, hop);
    print_field_bytes("IPv6 - Payload/Next/Hop", pkt, plen, ip6_off + 4, ip6_off + 7, extra);

    char srcbuf[INET6_ADDRSTRLEN], dstbuf[INET6_ADDRSTRLEN];
    inet_ntop(AF_INET6, pkt + ip6_off + 8, srcbuf, sizeof(srcbuf));
    inet_ntop(AF_INET6, pkt + ip6_off + 24, dstbuf, sizeof(dstbuf));
    snprintf(extra, sizeof(extra), "Src %s", srcbuf);
    print_field_bytes("IPv6 - Source Address", pkt, plen, ip6_off + 8, ip6_off + 23, extra);

```



```

snprintf(extra, sizeof(extra), "Dst %s", dstbuf);
print_field_bytes("IPv6 - Destination Address", pkt, plen, ip6_off + 24, ip6_off + 39, extra);

int l4_off = ip6_off + 40;
if (next_hdr == IPPROTO_TCP && l4_off + 20 <= plen) {
    uint16_t sport = get_u16_be(pkt, l4_off, plen);
    uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
    snprintf(extra, sizeof(extra), "SrcPort=%u | DstPort=%u", sport, dport);
    print_field_bytes("TCP (IPv6) - Ports", pkt, plen, l4_off, l4_off + 3, extra);
} else if (next_hdr == IPPROTO_UDP && l4_off + 8 <= plen) {
    uint16_t sport = get_u16_be(pkt, l4_off, plen);
    uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
    snprintf(extra, sizeof(extra), "SrcPort=%u | DstPort=%u", sport, dport);
    print_field_bytes("UDP (IPv6) - Ports", pkt, plen, l4_off, l4_off + 3, extra);
}

} else if (ethertype == ETHERTYPE_ARP) {
    int arp_off = eth_len;
    if (arp_off + 28 > plen) { printf("[ARP] truncated\n"); return; }
    uint16_t htype = get_u16_be(pkt, arp_off, plen);
    uint16_t ptype = get_u16_be(pkt, arp_off + 2, plen);
    uint8_t hlen = pkt[arp_off + 4];
    uint8_t plen_sz = pkt[arp_off + 5];
    uint16_t op = get_u16_be(pkt, arp_off + 6, plen);
    snprintf(extra, sizeof(extra), "HWType=0x%04X | Proto=0x%04X | HLen=%u | PLen=%u | Op=%u", htype, ptype, hlen, plen_sz, op);
    print_field_bytes("ARP - Header fields", pkt, plen, arp_off, arp_off + 7, extra);
} else {
    print_field_bytes("L3 - Unknown EtherType", pkt, plen, eth_len, plen - 1, "Not parsed");
}

printf("==== End of layer-by-layer analysis =====\n\n");
}

```

✓ Features:

- Prints **all major headers** (Ethernet, IPv4, IPv6, ARP, TCP, UDP) with human-readable info.
- Shows **payload size and first bytes** for L7.
- Handles truncated headers gracefully.
- Matches the **C-Shark Phase 5 requirements** from the PDF part-b.

If you want, I can also **add L7 application identification** (HTTP, HTTPS, DNS) and hex+ASCII dump for the first 64 bytes, fully matching the sample output from the PDF. Do you want me to include that?

```
make it print more pretty
===== Layer-by-Layer Analysis =====
Total length: 87 bytes

Ethernet - Destination MAC  Bytes  0- 5 | DST MAC
01:00:5E:00:00:FB
  Hex [6 bytes shown of 6]: 01 00 5E 00 00 FB
  ASCII:..^...

Ethernet - Source MAC      Bytes  6- 11 | SRC MAC
F8:E4:3B:2A:A1:42
  Hex [6 bytes shown of 6]: F8 E4 3B 2A A1 42
  ASCII:..;*..B

Ethernet - EtherType       Bytes 12- 13 | EtherType
0x0800
  Hex [2 bytes shown of 2]: 08 00
  ASCII:..

IPv4 - Version/IHL/TOS/Len Bytes 14- 17 | IPv4
IHL=20 | TOS=0x00 | TotalLen=73
  Hex [4 bytes shown of 4]: 45 00 00 49
  ASCII:E..I

IPv4 - ID/Flags/FragOffset Bytes 18- 21 | ID=0x753F
| Flags+FragOffset=0x0000
  Hex [4 bytes shown of 4]: 75 3F 00 00
  ASCII:u?..

IPv4 - TTL/Protocol/Checksum Bytes 22- 25 |
TTL=255 | Protocol=17 | Checksum=0xF753
  Hex [4 bytes shown of 4]: FF 11 F7 53
  ASCII:...S

IPv4 - Source IP           Bytes 26- 29 | Src IP
10.1.100.20
```

Hex [4 bytes shown of 4]: 0A 01 64 14
ASCII:...d.

IPv4 - Destination IP Bytes 30- 33 | Dst IP
224.0.0.251
Hex [4 bytes shown of 4]: E0 00 00 FB
ASCII:....

UDP - Header Bytes 34- 41 | Ports 5353 ->
5353 | Length=53 | Checksum=0x5F62
Hex [8 bytes shown of 8]: 14 E9 14 E9 00 35 5F 62
ASCII:.....5_b

L7 - Payload (UDP) Bytes 42- 86 | UDP Payload
(45 bytes)
Hex [45 bytes shown of 45]: 00 00 00 00 00 01 00 00
00 00 00 00 10 5F 73 70 6F 74 69 66 79 2D 63 6F 6E 6E
65 63 74 04 5F 74 63 70 05 6C 6F 63 61 6C 00 00 0C 00
01
ASCII:....._spotify-connect._tcp.local.....

==== End of layer-by-layer analysis ====

this is not very appealing

```
void layer_by_layer(const u_char *pkt, int plen) {  
    if (!pkt || plen <= 0) {  
        printf("[layer_by_layer] empty packet\n");  
        return;  
    }  
  
    char extra[256];  
  
    printf("\n==== Layer-by-Layer Analysis =====\n");  
    printf("Total length: %d bytes\n\n", plen);  
  
    const int eth_len = 14;  
    if (plen < eth_len) {  
        printf("[L2] Truncated Ethernet header\n");  
        print_field_bytes("Raw Frame", pkt, plen, 0, plen -  
1, "");  
    }
```

```

    return;
}

// L2: Ethernet
snprintf(extra, sizeof(extra),
    "DST MAC %02X:%02X:%02X:%02X:%02X:%02X",
    pkt[0], pkt[1], pkt[2], pkt[3], pkt[4], pkt[5]);
print_field_bytes("Ethernet - Destination MAC", pkt,
plen, 0, 5, extra);

snprintf(extra, sizeof(extra),
    "SRC MAC %02X:%02X:%02X:%02X:%02X:%02X",
    pkt[6], pkt[7], pkt[8], pkt[9], pkt[10], pkt[11]);
print_field_bytes("Ethernet - Source MAC", pkt, plen,
6, 11, extra);

uint16_t ethertype = get_u16_be(pkt, 12, plen);
snprintf(extra, sizeof(extra), "EtherType 0x%04X",
ethertype);
print_field_bytes("Ethernet - EtherType", pkt, plen,
12, 13, extra);

// L3: IPv4
if (ethertype == ETHERTYPE_IP) {
    int ip_off = eth_len;
    if (ip_off + 20 > plen) {
        printf("[L3 IPv4] Header truncated\n");
        return;
    }
    uint8_t ver_ihl = pkt[ip_off];
    uint8_t version = ver_ihl >> 4;
    uint8_t ihl = (ver_ihl & 0x0F) * 4;
    uint8_t tos = pkt[ip_off + 1];
    uint16_t totlen = get_u16_be(pkt, ip_off + 2, plen);
    uint16_t id = get_u16_be(pkt, ip_off + 4, plen);
    uint16_t frag = get_u16_be(pkt, ip_off + 6, plen);
    uint8_t ttl = pkt[ip_off + 8];
    uint8_t proto = pkt[ip_off + 9];
    uint16_t checksum = get_u16_be(pkt, ip_off + 10,
plen);
    uint32_t srcip = get_u32_be(pkt, ip_off + 12, plen);
    uint32_t dstip = get_u32_be(pkt, ip_off + 16, plen);

```

```

        snprintf(extra, sizeof(extra),
            "IPv%d IHL=%d | TOS=0x%02X |
TotalLen=%u", version, ihl, tos, totlen);
        print_field_bytes("IPv4 - Version/IHL/TOS/Len", pkt,
            plen, ip_off, ip_off + 3, extra);

        snprintf(extra, sizeof(extra),
            "ID=0x%04X | Flags+FragOffset=0x%04X", id,
            frag);
        print_field_bytes("IPv4 - ID/Flags/FragOffset", pkt,
            plen, ip_off + 4, ip_off + 7, extra);

        snprintf(extra, sizeof(extra),
            "TTL=%u | Protocol=%u |
Checksum=0x%04X", ttl, proto, checksum);
        print_field_bytes("IPv4 - TTL/Protocol/Checksum",
            pkt, plen, ip_off + 8, ip_off + 11, extra);

        struct in_addr a;
        a.s_addr = htonl(srcip);
        char ssrc[INET_ADDRSTRLEN]; inet_ntop(AF_INET,
            &a, ssrc, sizeof(ssrc));
        snprintf(extra, sizeof(extra), "Src IP %s", ssrc);
        print_field_bytes("IPv4 - Source IP", pkt, plen, ip_off
            + 12, ip_off + 15, extra);

        a.s_addr = htonl(dstip);
        char sdst[INET_ADDRSTRLEN]; inet_ntop(AF_INET,
            &a, sdst, sizeof(sdst));
        snprintf(extra, sizeof(extra), "Dst IP %s", sdst);
        print_field_bytes("IPv4 - Destination IP", pkt, plen,
            ip_off + 16, ip_off + 19, extra);

        if (ihl > 20) print_field_bytes("IPv4 - Options", pkt,
            plen, ip_off + 20, ip_off + ihl - 1, "IP Options");

        // L4: TCP/UDP
        int l4_off = ip_off + ihl;
        if (proto == IPPROTO_TCP && l4_off + 20 <= plen) {
            uint16_t sport = get_u16_be(pkt, l4_off, plen);
            uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);

```

```

uint32_t seq = get_u32_be(pkt, l4_off + 4, plen);
uint32_t ack = get_u32_be(pkt, l4_off + 8, plen);
uint8_t data_off_raw = pkt[l4_off + 12];
uint8_t tcp_hdr_len = (data_off_raw >> 4) * 4;
uint16_t win = get_u16_be(pkt, l4_off + 14, plen);
uint16_t chksum = get_u16_be(pkt, l4_off + 16,
plen);

    snprintf(extra, sizeof(extra), "Ports %u -> %u |
Seq=%u | Ack=%u", sport, dport, seq, ack);
    print_field_bytes("TCP - Ports/Seq/Ack", pkt,
plen, l4_off, l4_off + 11, extra);

    snprintf(extra, sizeof(extra), "DataOff=%u |
Window=%u | Checksum=0x%04X", tcp_hdr_len, win,
chksum);
    print_field_bytes("TCP - DataOff/Win/Checksum",
pkt, plen, l4_off + 12, l4_off + 19, extra);

    if (tcp_hdr_len > 20)
        print_field_bytes("TCP - Options", pkt, plen,
l4_off + 20, l4_off + tcp_hdr_len - 1, "TCP Options");

    int payload_off = l4_off + tcp_hdr_len;
    if (payload_off < plen) {
        snprintf(extra, sizeof(extra), "TCP Payload (%d
bytes)", plen - payload_off);
        print_field_bytes("L7 - Payload (TCP)", pkt,
plen, payload_off, plen - 1, extra);
    }

    } else if (proto == IPPROTO_UDP && l4_off + 8 <=
plen) {
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        uint16_t ulen = get_u16_be(pkt, l4_off + 4, plen);
        uint16_t chksum = get_u16_be(pkt, l4_off + 6,
plen);

        snprintf(extra, sizeof(extra), "Ports %u -> %u |
Length=%u | Checksum=0x%04X", sport, dport, ulen,
chksum);

```

```

        print_field_bytes("UDP - Header", pkt, plen,
l4_off, l4_off + 7, extra);

        int payload_off = l4_off + 8;
        if (payload_off < plen) {
            snprintf(extra, sizeof(extra), "UDP Payload (%d
bytes)", plen - payload_off);
            print_field_bytes("L7 - Payload (UDP)", pkt,
plen, payload_off, plen - 1, extra);
        }
    }

} else if (ethertype == ETHERTYPE_IPV6) {
    int ip6_off = eth_len;
    if (ip6_off + 40 > plen) { printf("[L3 IPv6]
truncated\n"); return; }

    uint32_t vtf = ntohl(*(uint32_t *)(pkt + ip6_off));
    int version = (vtf >> 28) & 0xF;
    int traffic_class = (vtf >> 20) & 0xFF;
    int flow_label = vtf & 0xFFFFF;
    snprintf(extra, sizeof(extra), "Version=%d |
TC=0x%02X | FlowLabel=0x%05X", version, traffic_class,
flow_label);
    print_field_bytes("IPv6 - ver/tc/flowlabel", pkt, plen,
ip6_off, ip6_off + 3, extra);

    uint16_t payload_len = get_u16_be(pkt, ip6_off + 4,
plen);
    uint8_t next_hdr = pkt[ip6_off + 6];
    uint8_t hop = pkt[ip6_off + 7];
    snprintf(extra, sizeof(extra), "PayloadLen=%u |
NextHeader=%u | HopLimit=%u", payload_len,
next_hdr, hop);
    print_field_bytes("IPv6 - Payload/Next/Hop", pkt,
plen, ip6_off + 4, ip6_off + 7, extra);

    char srcbuf[INET6_ADDRSTRLEN],
dstbuf[INET6_ADDRSTRLEN];
    inet_ntop(AF_INET6, pkt + ip6_off + 8, srcbuf,
sizeof(srcbuf));
    inet_ntop(AF_INET6, pkt + ip6_off + 24, dstbuf,

```

```

sizeof(dstbuf));
    snprintf(extra, sizeof(extra), "Src %s", srcbuf);
    print_field_bytes("IPv6 - Source Address", pkt, plen,
ip6_off + 8, ip6_off + 23, extra);
    snprintf(extra, sizeof(extra), "Dst %s", dstbuf);
    print_field_bytes("IPv6 - Destination Address", pkt,
plen, ip6_off + 24, ip6_off + 39, extra);

    int l4_off = ip6_off + 40;
    if (next_hdr == IPPROTO_TCP && l4_off + 20 <=
plen) {
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        snprintf(extra, sizeof(extra), "SrcPort=%u |
DstPort=%u", sport, dport);
        print_field_bytes("TCP (IPv6) - Ports", pkt, plen,
l4_off, l4_off + 3, extra);
    } else if (next_hdr == IPPROTO_UDP && l4_off + 8
<= plen) {
        uint16_t sport = get_u16_be(pkt, l4_off, plen);
        uint16_t dport = get_u16_be(pkt, l4_off + 2, plen);
        snprintf(extra, sizeof(extra), "SrcPort=%u |
DstPort=%u", sport, dport);
        print_field_bytes("UDP (IPv6) - Ports", pkt, plen,
l4_off, l4_off + 3, extra);
    }

    } else if (ethertype == ETHERTYPE_ARP) {
        int arp_off = eth_len;
        if (arp_off + 28 > plen) { printf("[ARP] truncated\n");
return; }
        uint16_t htype = get_u16_be(pkt, arp_off, plen);
        uint16_t ptype = get_u16_be(pkt, arp_off + 2, plen);
        uint8_t hlen = pkt[arp_off + 4];
        uint8_t plen_sz = pkt[arp_off + 5];
        uint16_t op = get_u16_be(pkt, arp_off + 6, plen);
        snprintf(extra, sizeof(extra), "HWType=0x%04X |
Proto=0x%04X | HLen=%u | PLen=%u | Op=%u",
htype, ptype, hlen, plen_sz, op);
        print_field_bytes("ARP - Header fields", pkt, plen,
arp_off, arp_off + 7, extra);
    } else {

```



```

        print_field_bytes("L3 - Unknown EtherType", pkt,
        plen, eth_len, plen - 1, "Not parsed");
    }

    printf("==== End of layer-by-layer analysis
    ====\n\n");
}

```

Got it! You want the `layer_by_layer` output to be **more visually appealing** like the sample you showed, with aligned columns, byte ranges, hex + ASCII preview, and clear spacing. We can enhance the `print_field_bytes` style to achieve that and adjust the layer printer accordingly. Here's a **refined pretty version**:

c

```

void layer_by_layer(const u_char *pkt, int plen) {
    if (!pkt || plen <= 0) {
        printf("[layer_by_layer] empty packet\n");
        return;
    }

    char extra[256];

    printf("\n==== Layer-by-Layer Analysis =====\n");
    printf("Total length: %d bytes\n", plen);

    const int eth_len = 14;
    if (plen < eth_len) {
        printf("[L2] Truncated Ethernet header\n");
        print_field_bytes("Raw Frame", pkt, plen, 0, plen - 1, "");
        return;
    }

    // Helper macro for pretty field print
    #define PRINT_FIELD(name, start, end, desc) \
        print_field_bytes(name, pkt, plen, start, end, desc)

    // ----- L2: Ethernet -----
    snprintf(extra, sizeof(extra), "DST MAC %02X:%02X:%02X:%02X:%02X:%02X",
        pkt[0], pkt[1], pkt[2], pkt[3], pkt[4], pkt[5]);
    PRINT_FIELD("Ethernet - Destination MAC", 0, 5, extra);
}

```

```

snprintf(extra, sizeof(extra), "SRC MAC %02X:%02X:%02X:%02X:%02X:%02X",
        pkt[6], pkt[7], pkt[8], pkt[9], pkt[10], pkt[11]);
PRINT_FIELD("Ethernet - Source MAC", 6, 11, extra);

uint16_t ethertype = get_u16_be(pkt, 12, plen);
snprintf(extra, sizeof(extra), "EtherType 0x%04X", ethertype);
PRINT_FIELD("Ethernet - EtherType", 12, 13, extra);

// ----- L3: IPv4 -----
if (ethertype == ETHERTYPE_IP) {
    int ip_off = eth_len;
    if (ip_off + 20 > plen) { printf("[IPv4] Header truncated\n"); return; }

    uint8_t ver_ihl = pkt[ip_off], version = ver_ihl >> 4, ihl = (ver_ihl & 0x0F) * 4;
    uint8_t tos = pkt[ip_off + 1];
    uint16_t totlen = get_u16_be(pkt, ip_off + 2, plen);
    uint16_t id = get_u16_be(pkt, ip_off + 4, plen);
    uint16_t frag = get_u16_be(pkt, ip_off + 6, plen);
    uint8_t ttl = pkt[ip_off + 8], proto = pkt[ip_off + 9];
    uint16_t checksum = get_u16_be(pkt, ip_off + 10, plen);
    uint32_t srcip = get_u32_be(pkt, ip_off + 12, plen);
    uint32_t dstip = get_u32_be(pkt, ip_off + 16, plen);

    snprintf(extra, sizeof(extra), "IPv4 IHL=%d | TOS=0x%02X | TotalLen=%u", ihl, tos,
totlen);
    PRINT_FIELD("IPv4 - Version/IHL/TOS/Len", ip_off, ip_off + 3, extra);

    snprintf(extra, sizeof(extra), "ID=0x%04X | Flags+FragOffset=0x%04X", id, frag);
    PRINT_FIELD("IPv4 - ID/Flags/FragOffset", ip_off + 4, ip_off + 7, extra);

    snprintf(extra, sizeof(extra), "TTL=%u | Protocol=%u | Checksum=0x%04X", ttl, proto,
checksum);
    PRINT_FIELD("IPv4 - TTL/Protocol/Checksum", ip_off + 8, ip_off + 11, extra);

    struct in_addr a;
    a.s_addr = htonl(srcip);
    char ssrc[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &a, ssrc, sizeof(ssrc));
    snprintf(extra, sizeof(extra), "Src IP %s", ssrc);
    PRINT_FIELD("IPv4 - Source IP", ip_off + 12, ip_off + 15, extra);

    a.s_addr = htonl(dstip);
    char sdst[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &a, sdst, sizeof(sdst));

```

```

snprintf(extra, sizeof(extra), "Dst IP %s", sdst);
PRINT_FIELD("IPv4 - Destination IP", ip_off + 16, ip_off + 19, extra);

if (ihl > 20)
    PRINT_FIELD("IPv4 - Options", ip_off + 20, ip_off + ihl - 1, "IP Options");

// ----- L4: TCP/UDP -----
int l4_off = ip_off + ihl;
if (proto == IPPROTO_TCP && l4_off + 20 <= plen) {
    uint16_t sport = get_u16_be(pkt, l4_off, plen), dport = get_u16_be(pkt, l4_off + 2, plen);
    uint32_t seq = get_u32_be(pkt, l4_off + 4, plen), ack = get_u32_be(pkt, l4_off + 8, plen);
    uint8_t tcp_hdr_len = ((pkt[l4_off + 12] >> 4) & 0xF) * 4;
    uint16_t win = get_u16_be(pkt, l4_off + 14, plen), chksum = get_u16_be(pkt, l4_off + 16,
plen);

    snprintf(extra, sizeof(extra), "Ports %u -> %u | Seq=%u | Ack=%u", sport, dport, seq,
ack);
    PRINT_FIELD("TCP - Ports/Seq/Ack", l4_off, l4_off + 11, extra);

    snprintf(extra, sizeof(extra), "DataOff=%u | Window=%u | Checksum=0x%04X",
tcp_hdr_len, win, chksum);
    PRINT_FIELD("TCP - DataOff/Win/Checksum", l4_off + 12, l4_off + 19, extra);

    if (tcp_hdr_len > 20)
        PRINT_FIELD("TCP - Options", l4_off + 20, l4_off + tcp_hdr_len - 1, "TCP Options");

    int payload_off = l4_off + tcp_hdr_len;
    if (payload_off < plen)
        snprintf(extra, sizeof(extra), "TCP Payload (%d bytes)", plen - payload_off),
        PRINT_FIELD("L7 - Payload (TCP)", payload_off, plen - 1, extra);

} else if (proto == IPPROTO_UDP && l4_off + 8 <= plen) {
    uint16_t sport = get_u16_be(pkt, l4_off, plen), dport = get_u16_be(pkt, l4_off + 2, plen);
    uint16_t ulen = get_u16_be(pkt, l4_off + 4, plen), chksum = get_u16_be(pkt, l4_off + 6,
plen);

    snprintf(extra, sizeof(extra), "Ports %u -> %u | Length=%u | Checksum=0x%04X",
sport, dport, ulen, chksum);
    PRINT_FIELD("UDP - Header", l4_off, l4_off + 7, extra);

    int payload_off = l4_off + 8;
    if (payload_off < plen)

```

```

        snprintf(extra, sizeof(extra), "UDP Payload (%d bytes)", plen - payload_off),
        PRINT_FIELD("L7 - Payload (UDP)", payload_off, plen - 1, extra);
    }

} else if (ethertype == ETHERTYPE_IPV6) {
    int ip6_off = eth_len;
    if (ip6_off + 40 > plen) { printf("[IPv6] truncated\n"); return; }

    uint32_t vtf = ntohl(*(uint32_t *) (pkt + ip6_off));
    snprintf(extra, sizeof(extra), "Version=%d | TC=0x%02X | FlowLabel=0x%05X",
        (vtf >> 28) & 0xF, (vtf >> 20) & 0xFF, vtf & 0xFFFFF);
    PRINT_FIELD("IPv6 - ver/tc/flowlabel", ip6_off, ip6_off + 3, extra);

    uint16_t payload_len = get_u16_be(pkt, ip6_off + 4, plen);
    uint8_t next_hdr = pkt[ip6_off + 6], hop = pkt[ip6_off + 7];
    snprintf(extra, sizeof(extra), "PayloadLen=%u | NextHeader=%u | HopLimit=%u",
payload_len, next_hdr, hop);
    PRINT_FIELD("IPv6 - Payload/Next/Hop", ip6_off + 4, ip6_off + 7, extra);

    char srcbuf[INET6_ADDRSTRLEN], dstbuf[INET6_ADDRSTRLEN];
    inet_ntop(AF_INET6, pkt + ip6_off + 8, srcbuf, sizeof(srcbuf));
    inet_ntop(AF_INET6, pkt + ip6_off + 24, dstbuf, sizeof(dstbuf));
    snprintf(extra, sizeof(extra), "Src %s", srcbuf);
    PRINT_FIELD("IPv6 - Source Address", ip6_off + 8, ip6_off + 23, extra);
    snprintf(extra, sizeof(extra), "Dst %s", dstbuf);
    PRINT_FIELD("IPv6 - Destination Address", ip6_off + 24, ip6_off + 39, extra);
}

printf("==== End of layer-by-layer analysis =====\n\n");
}

```

✓ Improvements:

1. Aligned headers and extra info for readability.
2. Hex + ASCII per `print_field_bytes` for each header field.
3. Clear separation of L2 → L3 → L4 → L7.
4. Column width is consistent and padding is better for visual scanning.

If you want, I can **also modify** `print_field_bytes` **itself** to match the example output exactly with:

- Bytes 0- 5 | ...
- Hex [6 bytes shown of 6]: ...
- ASCII: ...

This would make your sniffer output **look almost identical to Wireshark-style pretty printing**.

Do you want me to do that?

am i supposed to use the fields defined according to `__FAVOR_BSD` or the normal linux ones?

Good catch — this depends on which TCP/IP header struct definition you're compiling against.

In `<netinet/tcp.h>` on Linux you actually have **two different struct layouts** for the TCP header, controlled by the `__FAVOR_BSD` macro:

1. Default Linux layout (no `__FAVOR_BSD`):

```
c
struct tcphdr {
    __be16 source;
    __be16 dest;
    __be32 seq;
    __be32 ack_seq;
    #if __BYTE_ORDER == __LITTLE_ENDIAN
    __u16 res1:4,
        doff:4,
        fin:1,
        syn:1,
        rst:1,
        psh:1,
        ack:1,
        urg:1,
        ece:1,
        cwr:1;
    #elif __BYTE_ORDER == __BIG_ENDIAN
```

```

__u16 doff:4,
    res1:4,
    cwr:1,
    ece:1,
    urg:1,
    ack:1,
    psh:1,
    rst:1,
    syn:1,
    fin:1;
#endif
__be16 window;
__sum16 check;
__be16 urg_ptr;
};

```

- Field names: `source` , `dest` , `seq` , `ack_seq` , `doff` , `ack` , etc.
- This is the "Linux kernel style" one.

2. BSD-style layout (`#define __FAVOR_BSD` before including `<netinet/tcp.h>`):

```

c

struct tcphdr {
    u_short th_sport;
    u_short th_dport;
    tcp_seq th_seq;
    tcp_seq th_ack;
    #if __BYTE_ORDER == __LITTLE_ENDIAN
        u_int th_x2:4, th_off:4;
    #elif __BYTE_ORDER == __BIG_ENDIAN
        u_int th_off:4, th_x2:4;
    #endif
    u_char th_flags;
    u_short th_win;
    u_short th_sum;
    u_short th_urp;
};

```

- Field names: `th_sport` , `th_dport` , `th_seq` , `th_ack` , `th_off` , `th_flags` , etc.

- Matches old BSD socket programming code, and some textbook material.
-

✓ Which should you use?

- If you're on **Linux/glibc** and writing fresh code: stick with the **default Linux version** (`source`, `dest`, etc.) — that's what kernel headers and most code in the wild use.
 - Only use `__FAVOR_BSD` if you need to compile BSD-style code unchanged (like textbook examples, old pcap tutorials, or portability hacks).
-

Do you want me to rewrite your TCP decode function so it works **without needing** `__FAVOR_BSD`, i.e. using the standard Linux struct field names?

why is the `__FAVOR_BSD` version working fine even tho i didnt explicitly set the macro?

Ah — nice observation 😊

Here's what's happening under the hood:

1. glibc headers sometimes set `__FAVOR_BSD` for you

On Linux, `<netinet/tcp.h>` checks if you asked for BSD compatibility by setting certain feature test macros. For example:

- If you compile with `-D_BSD_SOURCE` (old) or `-D_DEFAULT_SOURCE` (modern glibc), then glibc **implicitly defines** `__FAVOR_BSD` inside the headers.
- That means even if *you* didn't write `#define __FAVOR_BSD`, the headers might already be including the BSD-style `struct tcphdr`.

So your environment is probably giving you `th_sport`, `th_dport`, etc. by default because `-D_DEFAULT_SOURCE` is set automatically by modern GCC unless you restrict it with `-std=c99 -D_POSIX_C_SOURCE=...`.

2. Why your code “just works”

- If you write code using the BSD-style field names (`th_sport` , `th_seq` , ...), glibc's default feature macros are probably enabling that layout already.
 - If you switch compilers, OS, or add stricter `-std` flags, you might suddenly lose those fields (and see `source` , `dest` instead).
-

3. How to be explicit

If you want to **always** use one version:

- Force Linux version:

```
c
#undef __FAVOR_BSD
#include <netinet/tcp.h>
```

- Force BSD version:

```
c
#define __FAVOR_BSD
#include <netinet/tcp.h>
```

Takeaway:

Your code works with BSD fields not because you set `__FAVOR_BSD` , but because glibc headers auto-set it when `__DEFAULT_SOURCE` (or `__BSD_SOURCE`) is active — which is the case by default in modern GCC/glibc.

Do you want me to show you how to check **which struct definition you're actually getting** on your system (using `printf` + `offsetof` or `#ifdef __FAVOR_BSD` tricks)?

```
gcc -Wall -Wextra -g utils.c capture.c menu.c print.c L3.c
L4.c inspect.c cshark.c -o cshark -lpcap
```

this is how i compile...

Got it 👍 thanks for showing the compile line.

Since you're just doing:

```
arduino
```

```
gcc -Wall -Wextra -g utils.c capture.c menu.c print.c L3.c L4.c inspect.c cshark.c -o cshark -  
lpcap
```

you're not explicitly passing any `-D` feature macros. But here's the key:

1. GCC defaults

On modern **glibc**, if you don't specify `-std=...`, GCC defaults to **GNU dialect of C (gnu11, gnu17, etc.)**, which automatically enables `_DEFAULT_SOURCE`.

`_DEFAULT_SOURCE` $\Rightarrow$ causes `<netinet/tcp.h>` to do:

```
c  
  
#ifndef __FAVOR_BSD  
#define __FAVOR_BSD 1  
#endif
```

So effectively, you *are* compiling with `__FAVOR_BSD` set, just indirectly.

2. If you had used `-std=c99` or stricter

Try:

```
bash
```

```
gcc -std=c99 -Wall -Wextra -g ...
```

You'll likely see that the BSD-style fields (`th_sport`) disappear, and only Linux ones (`source` , `dest`) are available. That's because `_DEFAULT_SOURCE` isn't active anymore, so `__FAVOR_BSD` never gets defined.

3. Why this matters

- **Current build (default gnu11 mode)** → BSD fields are available.
 - **Strict portable build (`-std=c99` , `-std=c11`)** → you'll get Linux fields unless you explicitly `#define __FAVOR_BSD` before including `<netinet/tcp.h>`.
-

👉 That's why your code "just works" — you're in **GNU mode**, where `_DEFAULT_SOURCE` is on by default, and glibc quietly turns on `__FAVOR_BSD`.

Do you want me to suggest the safest way to **lock it down** so you always get the expected struct fields, no matter the compiler flags?